

`pd.Series()` создаёт одномерную индексированную структуру Pandas, а параметр `name` задаёт имя серии.

`pd.DataFrame()` формирует таблицу, в данном примере ей передаётся словарь, где ключи становятся именами столбцов, а списки — их значениями.

Цепочка `frame.groupby("type")["value"].sum()` сначала группирует строки по значениям столбца `type`, затем выбирает столбец `value` и вычисляет сумму внутри каждой группы.

```
s = pd.Series([10, 20, 30], name="value")
frame = pd.DataFrame({
    "type": ["A", "B", "A"],
    "value": [10, 20, 30]
})

print(type(s).__name__)
print(type(frame).__name__)
print(frame.groupby("type")["value"].sum())
```

Результат выполнения

```
Series
DataFrame
type
A      40
B      20
Name: value, dtype: int64
```

`Series` — один именованный столбец с индексом; `DataFrame` — набор согласованных `Series`. `groupby()` реализует типичную аналитическую операцию «разбить на группы → агрегировать».

4. Форматы файлового хранения данных и загрузка CSV

CSV — текстовый табличный формат без строгой схемы типов: значения хранятся строками, а типы восстанавливаются при чтении. JSON удобен для вложенных и полуструктурированных объектов, но при записи табличных данных часто занимает больше места. Parquet — бинарный колоночный формат, сохраняющий типы и ориентированный на аналитическое чтение отдельных столбцов. В последующих работах Spark будет особенно эффективно использовать Parquet.

Таблица 2. Популярные форматы файлового хранения данных

Формат	Организация	Схема	Сильная сторона	Ограничение
CSV	Строки/разделители	Нет встроенной строгой схемы	Высокая совместимость, просмотр текстом	Повторное определение типов; чтение лишних столбцов.
JSON	Объекты/массивы	Типы выражены структурой JSON	Гибкая вложенная структура	Избыточность ключей и большой размер для таблиц.
Parquet	Колоночный бинарный	Да	Сжатие, сохранение типов, выборочное чтение столбцов	Нужен специальный reader/engine.

Практический блок 4.1. Загрузка FiresRu.csv

В блоке выполним первичную проверку самого факта корректной загрузки данных. Сначала создадим путь к FiresRu.csv, затем считаем содержимое файла в объект `df`. После чтения проверим размер таблицы и состав полей, и с помощью `head()` покажем несколько реальных наблюдений.

Класс `Path` из стандартного модуля `pathlib` представляет путь к файлу как объект и позволяет не связывать код с особенностями строковых путей конкретной операционной системы.

Функция `pd.read_csv()` читает CSV и возвращает `DataFrame`. Атрибут `df.shape` содержит кортеж пары (число строк, число столбцов), `df.columns` — индекс с именами столбцов, а `.tolist()` преобразует его в обычный список Python.

Метод `head()` возвращает первые пять строк и используется для быстрой визуальной проверки структуры загруженной таблицы.

Задача состоит в том, чтобы убедиться, что файл прочитан как ожидаемая таблица и что дальнейший анализ будет выполняться над правильным объектом.

```
from pathlib import Path
import pandas as pd
```

```
DATA_PATH = Path("FiresRu.csv")
df = pd.read_csv(DATA_PATH)

print(df.shape)
print(df.columns.tolist())
df.head()
```

Результат выполнения

```
(26681, 9)
['type_name', 'type_id', 'lon', 'lat', 'temperature_c',
'precipitation_mm', 'relative_humidity', 'wind_speed_ms',
'solar_radiation']
```

Таблица 3. Результат выполнения блока 4.1. в виде таблицы

type_name	type_id	lon	lat	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	solar_radiation
Natural fire	4	131.59	47.87	-9.44	0.01	72.98	1.14	4.59
Natural fire	4	131.59	47.88	-9.44	0.01	72.98	1.14	4.59
Forest fire	3	131.99	48.50	-10.71	0.05	81.31	0.80	4.24
Natural fire	4	131.90	43.63	-7.77	0.00	68.41	0.97	4.66
Natural fire	4	131.57	47.86	-9.44	0.01	72.98	1.14	4.59

Загрузка прошла без явного задания `sep` и `encoding`: файл использует стандартный разделитель «,» и читается как таблица из 26 681 строк и 9 столбцов. По перечню полей сразу видно отсутствие временной метки — значит, временной анализ на этом наборе не является корректной задачей.

5. Первичная диагностика данных

Первичный анализ до построения графиков и вычисления сложных метрик ставит своей целью выяснить какой размер у входной таблицы данных, какие типы данных в атрибутах распознаны, есть ли пропуски и дубликаты в данных, какие диапазоны значений наблюдаются.

Метод `head()` показывает примеры строк, `info()` — структуру и память, `describe()` — основные статистики, `isna()` и `duplicated()` — простейшие индикаторы качества.

Практический блок 5.1. Структура и память

Вместо непосредственного вывода `info()` в интерфейс Notebook его результат будет сначала перехвачен в строковый буфер, а затем напечатан. Это удобно для